

Date _____

Semantic Analyzer Design Melody Script - DSL

Introduction: The Semantic Analyzer (Phase 3) ensures that program is logically consistent. It performs type checking and scope management using a Symbol Table. This document illustrates how the symbol table is populated and used during the compilation of a sample MelodyScript program.

Sample Program.

Consider the following MelodyScript code snippet.

```
int bpm = 120;  
note root = C4;  
repeat (4) {  
    int offset = 2;  
    note current = root + offset;  
    play (current, 500);  
}
```

Filled Symbol Table: The Symbol table is organized by scopes. In this example there is a Global scope and a Local Scope (inside the repeat block).

3.1. Global Scope

Date _____

Name	Type	Value (at compile Time)	Scope
bpm	int	120	Global
root	note	C ₄ (freq = 262)	Global

3.2 Local Scope (Inside repeat block)

Name	Type	Value (at compile Time)	Scope
offset	int	2	local (Repeat Block)
current	note	Result of root + offset	Local (repeat Block)

4. Semantic Validation Process

During the Analysis of Sample Program, the Semantic Analyzer performs the following checks:

1. Variable Declaration: when `int bpm = 120;` is encountered, the Analyzer adds bpm to the current (global) Scope. If bpm was already there, a compiler Error would be raised.

2. Type Consistency:

In note `current = root + offset;` the analyzer looks up root in the Symbol table (found in Global, type int)

Date _____

- looks up offset in the symbol table
(found in local, type int)
- validates the + operator for note and int (valid for transposition)
- confirms the resulting type is note, which matches the declaration of current.

3. Function Argument validation.

For Play (current, ~~500~~); the analyzer:

- confirms play is a built in function.
- checks that current is note.
- checks that 500 is an note ~~int~~.

4. Scope Exit: When the repeat block ends, the analyzer pops the local scope, making offset and current inaccessible to subsequent code.

5. Type matrix: the following table summarizes the allowed operations and their resulting types.

Left type	operator	Right Type	Result Type	<u>note</u>
<u>int</u>	<u>+</u> , <u>-</u> , <u>*</u> , <u>/</u>	<u>int</u>	<u>int</u>	standard arithmetic
<u>note</u>	<u>+</u> , <u>-</u> , <u>*</u> , <u>/</u>	<u>int</u>	<u>note</u>	Transposition (shifting pitch)

Left Type	operator	Right Type	Result Type	Note
int	$>, <, =$	int	int (Bool)	Comparison resulting 0 or 1
node	$\equiv$	node	int (Bool)	checking if two nodes are identical.